



Portfolio Demonstration: Authorized Web Application Security Assessment

Target: OWASP Juice Shop (Intentionally Vulnerable Training Lab)

Prepared By: Anyaugo Space-Tech LTD

Contact: security@anyaugo.llc

Assessment Type: Web Application & API Security Assessment

Methodology: OWASP Web Security Testing Guide (WSTG v4.2) & OWASP Top 10:2025

Document Classification: Public Portfolio Sample / Non-Client Demonstration

Prepared For: Portfolio Review / Technical Assessment

Portfolio Disclaimer:

This security assessment report is a technical portfolio demonstration based on OWASP Juice Shop, an intentionally vulnerable open-source web application created for legal security training. This document does not represent an assessment of a real-world enterprise system. Its purpose is to demonstrate security assessment methodology, evidence collection, risk analysis, business impact communication, remediation guidance, and retest planning.

1. Executive Summary

An unauthenticated security assessment was performed against the OWASP Juice Shop lab application. The objectives were to evaluate the application's attack surface, identify security weaknesses, explain business impact, and provide actionable remediation guidance.

The assessment identified five representative findings across authentication, access control, input handling, and configuration management. In a real environment, similar weaknesses could create risk around account access, sensitive files, client-side script execution, API discovery, and access to non-owned customer resources.

2. Scope And Methodology

- **Application Target:** OWASP Juice Shop (<http://localhost:3000>)
- **Environment:** Isolated local lab environment
- **Scope Surface:** Public web frontend, REST API behavior, static file endpoints, and browser-side storage mechanisms
- **Standards:** OWASP WSTG v4.2 and OWASP Top 10:2025
- **Scoring:** Severity levels are baseline estimates using CVSS concepts plus business impact. Production scores should be calibrated to the client's environment.

3. Tools Actually Used

Tool	Purpose
OWASP Juice Shop	Intentionally vulnerable local web application lab
Chrome Developer Tools	Network, Console, Application Storage, and XHR/Fetch inspection
Browser-based manual testing	Authenticated and unauthenticated request validation
Screenshot evidence collection	Finding documentation and proof capture
Codex	Report structuring, business impact wording, remediation guidance, and portfolio review

4. Findings Summary

ID	Finding	Severity	Mapping
Finding 1	SQL Injection - Authentication Bypass	Critical	OWASP Top 10:2025 A05 - Injection WSTG-INPV-05
Finding 2	Sensitive Data Exposure via Directory Listing	High	OWASP Top 10:2025 A01 - Broken Access Control WSTG-CONF-04
Finding 3	DOM-Based Cross-Site Scripting	High	OWASP Top 10:2025 A05 - Injection WSTG-CLNT-01
Finding 4	Information Disclosure via Exposed API Documentation	Medium	OWASP Top 10:2025 A02 - Security Misconfiguration WSTG-CONF-05
Finding 5	Insecure Direct Object Reference on User Baskets	High	OWASP Top 10:2025 A01 - Broken Access Control WSTG-ATHZ-04

5. Technical Findings And Vulnerability Analysis

Finding 1: SQL Injection - Authentication Bypass

Severity: Critical (9.8 baseline estimate)

Taxonomy: OWASP Top 10:2025 A05 - Injection | WSTG-INPV-05

Description: The user authentication endpoint failed to safely handle user-supplied input before database query evaluation, allowing authentication logic to be bypassed in the lab environment.

Proof of Concept:

- Navigated to `/api/Users/login`.
- Submitted a controlled SQL injection test string in the email field.
- Observed an HTTP 200 OK response with an administrative JSON Web Token in the lab environment.

Business Impact: Could allow an unauthenticated actor to bypass authentication and access privileged account functions, depending on database permissions and role controls.

Remediation Guidance: Use parameterized queries or ORM prepared statements, validate input server-side, and store session identifiers in HttpOnly and Secure cookies instead of browser-accessible local storage.

Evidence: See Appendix A for supporting screenshots.

Finding 2: Sensitive Data Exposure via Directory Listing

Severity: High (7.5 baseline estimate)

Taxonomy: OWASP Top 10:2025 A01 - Broken Access Control | WSTG-CONF-04

Description: Directory indexing was enabled on the public `/ftp` endpoint, allowing unauthenticated users to inspect directory contents and access internal documents in the lab application.

Proof of Concept:

- Navigated to `http://localhost:3000/ftp`.
- Accessed `acquisitions.md` from the exposed directory.
- Reviewed internal document contents in the controlled lab.

Business Impact: Inappropriate exposure of sensitive or proprietary documentation can increase disclosure risk and accelerate attacker reconnaissance.

Remediation Guidance: Disable directory browsing, remove sensitive files from public web paths, and enforce server-side access control on static assets that should not be public.

Evidence: See Appendix A for supporting screenshots.

Finding 3: DOM-Based Cross-Site Scripting

Severity: High (7.2 baseline estimate)

Taxonomy: OWASP Top 10:2025 A05 - Injection | WSTG-CLNT-01

Description: The search functionality reflected user-controlled input into the browser DOM without sufficient output encoding, allowing script execution in the controlled lab environment.

Proof of Concept:

- Navigated to `/#/search`.
- Submitted a controlled XSS test payload in the search field.
- Observed JavaScript execution through an alert dialog in the application origin.

Business Impact: Could support session manipulation, UI deception, or arbitrary client-side request execution if combined with exposed tokens or unsafe client-side storage.

Remediation Guidance: Apply context-aware output encoding, avoid unsafe DOM sinks, sanitize user-controlled input before rendering, and enforce a restrictive Content Security Policy.

Evidence: See Appendix A for supporting screenshots.

Finding 4: Information Disclosure via Exposed API Documentation

Severity: Medium (5.3 baseline estimate)

Taxonomy: OWASP Top 10:2025 A02 - Security Misconfiguration | WSTG-CONF-05

Description: The application exposed interactive Swagger/OpenAPI documentation at `/api-docs` without authentication in the lab environment.

Proof of Concept:

- Navigated to `http://localhost:3000/api-docs`.
- Reviewed exposed REST API endpoints, schemas, and operational parameters.

Business Impact: Public API documentation can accelerate reconnaissance by revealing endpoints, naming conventions, payload structures, and possible attack surfaces.

Remediation Guidance: Disable Swagger UI and schema exposure in production or restrict access with authentication, authorization, and environment-specific configuration.

Evidence: See Appendix A for supporting screenshots.

Finding 5: Insecure Direct Object Reference on User Baskets

Severity: High (8.5 baseline estimate)

Taxonomy: OWASP Top 10:2025 A01 - Broken Access Control | WSTG-ATHZ-04

Description: The `/rest/basket/{id}` API endpoint returned basket data based on a requested resource identifier without verifying whether the requesting user owned that resource.

Proof of Concept:

- Issued an authenticated request to `/rest/basket/2` from a different user's session in the lab.
- Observed an HTTP 200 OK response containing another user's basket data.

Business Impact: Could allow unauthorized access to private customer records or order activity across non-owned application accounts.

Remediation Guidance: Enforce server-side object-level authorization checks so every resource request validates ownership or explicit role permission before returning data.

Evidence: See Appendix A for supporting screenshots.

6. Remediation Verification And Retest Plan

Portfolio Lab Retest Method

For this portfolio demonstration, the retest plan describes how each identified vulnerability would be validated after remediation in the controlled OWASP Juice Shop lab environment.

- **SQL Injection:** Recheck the authentication endpoint and confirm the bypass condition no longer succeeds.
- **Directory Listing:** Access `/ftp` and `/ftp/acquisitions.md` and confirm access is blocked or restricted.
- **DOM-Based XSS:** Resubmit the test input and confirm it is encoded and rendered safely as text.
- **API Documentation Exposure:** Query `/api-docs` and confirm access is disabled or restricted in production-like configuration.
- **IDOR:** Request `/rest/basket/2` from a non-owner session and confirm the API rejects the request.

Client Engagement Retest Rules

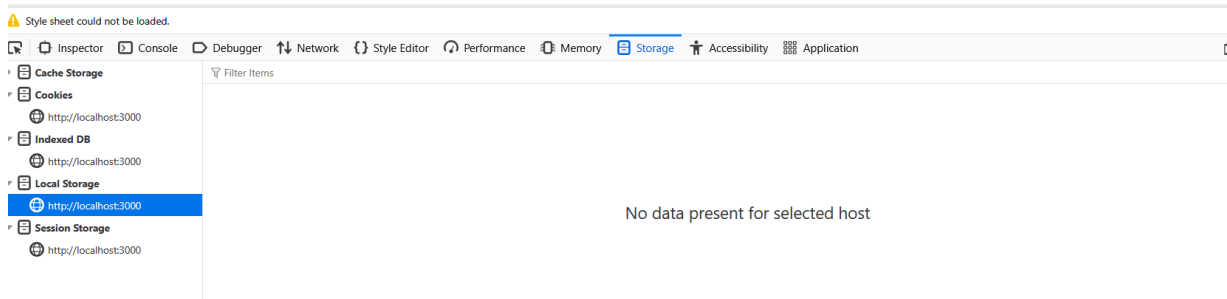
In a commercial client engagement, retesting is performed exclusively on authorized client assets after formal written authorization, an agreed scope, and confirmation that remediation steps have been deployed. Retesting validates whether originally reported vulnerabilities are resolved and concludes with a Remediation Verification Addendum.

Appendix A: Evidence Screenshots

The screenshots below support the five findings in this portfolio assessment. They are from a controlled OWASP Juice Shop lab and should not be interpreted as evidence against any real company system.

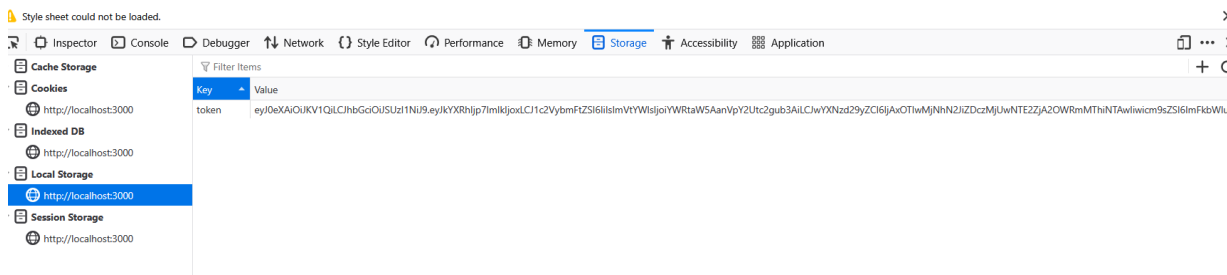
Finding 1: SQL Injection - Authentication Bypass

Evidence A1: Empty local storage before authentication



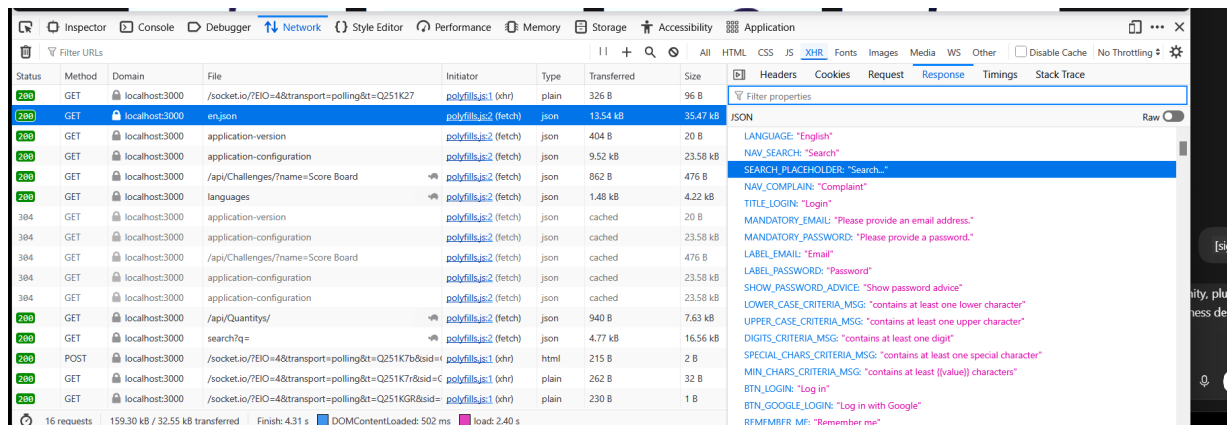
Source file: Test-1-SQL-Injection\Finding1A-Empty-Local-Storage.png

Evidence A2: Token populated in browser storage after authentication bypass



Source file: Test-1-SQL-Injection\Finding1B-Populated-Token-In-Local-Storage.png

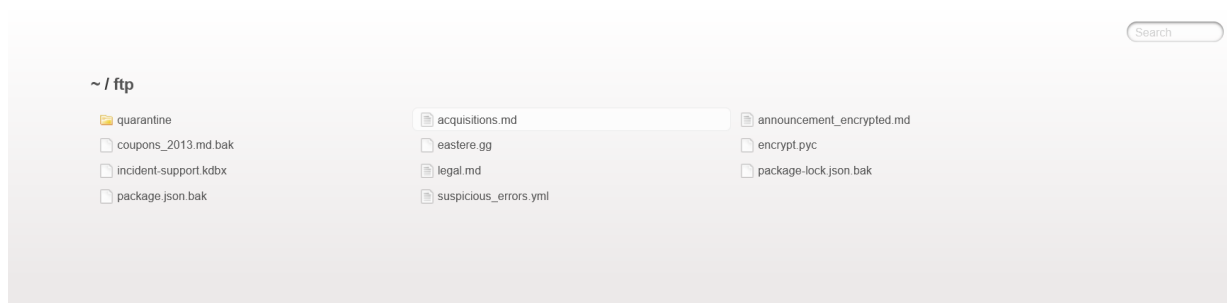
Evidence A3: XHR request and authentication response context



Source file: Test-1-SQL-Injection\Finding1C-XHR-Request.png

Finding 2: Sensitive Data Exposure via Directory Listing

Evidence B1: Public FTP-style directory listing



Source file: Test-2-Broken-Access-Control_Sensitive-Data-Exposure\Finding2A-FTP-Directory.png

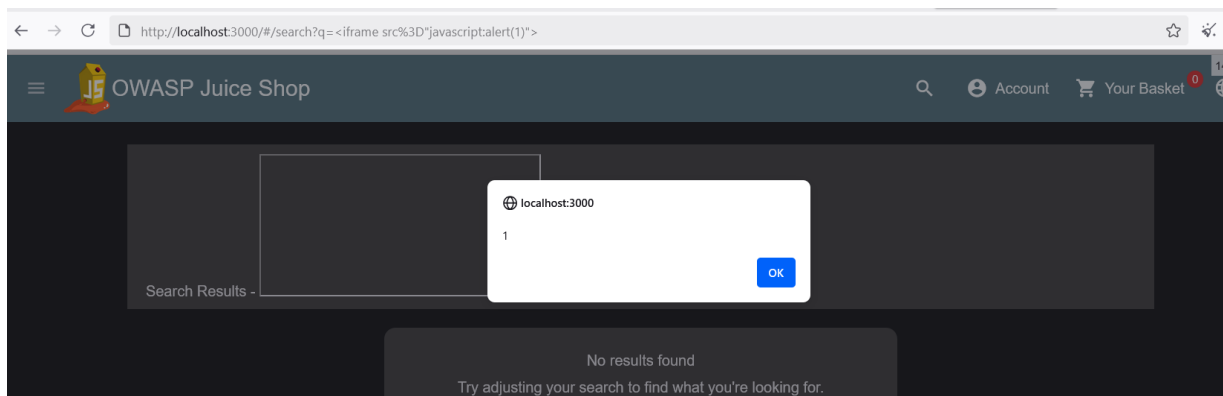
Evidence B2: Accessible internal document in the lab application



Source file: Test-2-Broken-Access-Control_Sensitive-Data-Exposure\Finding2B-Exfiltrated-Confidential-Document.png

Finding 3: DOM-Based Cross-Site Scripting

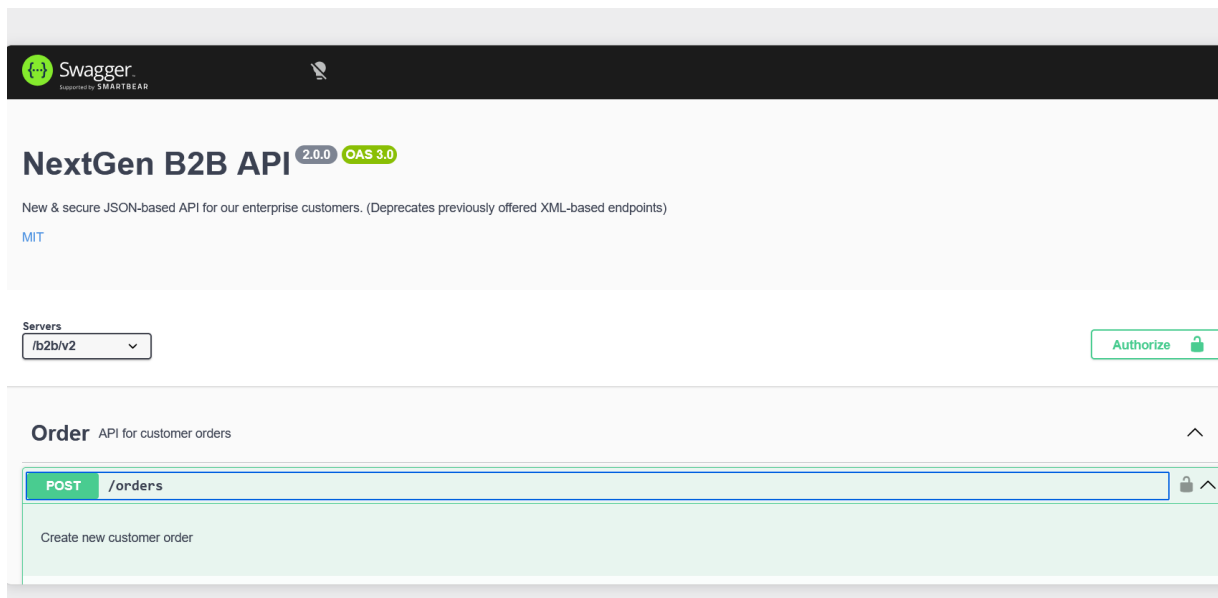
Evidence C1: DOM-based XSS execution proof in the lab application



Source file: Test-3-DOM-Based Reflected Cross-Site Scripting (XSS)\Finding3-XSS-Execution.png

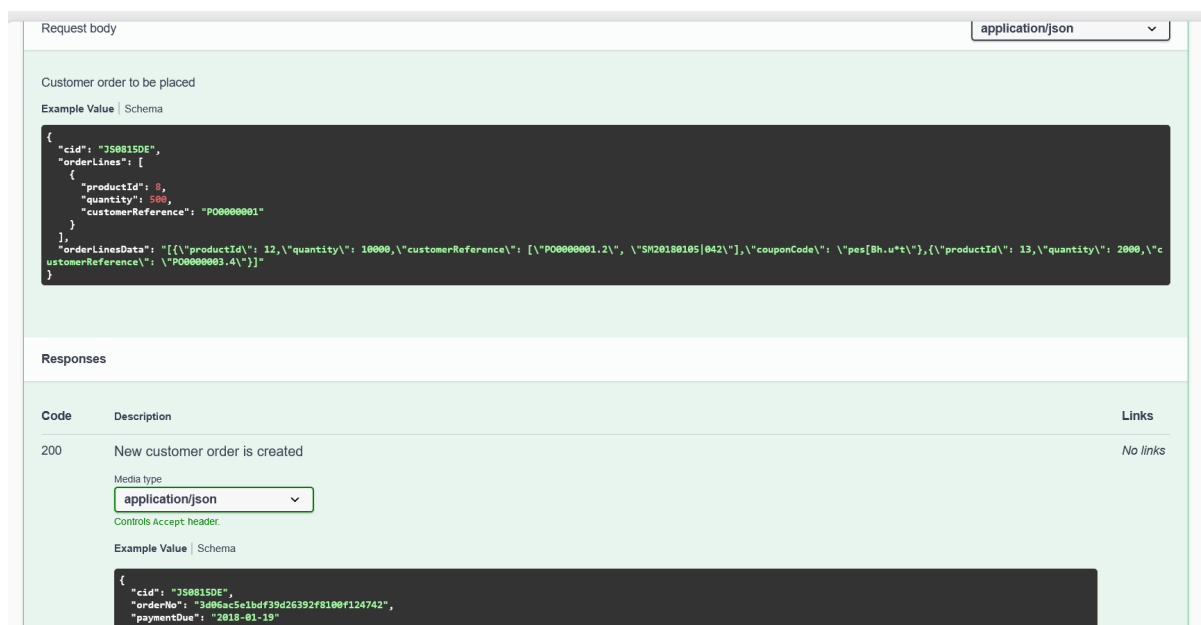
Finding 4: Information Disclosure via Exposed API Documentation

Evidence D1: Public Swagger/OpenAPI interface



Source file: Test-4-Information-Disclosure-via-Exposed-Endpoint\Finding4A-Swagger-Interface.png

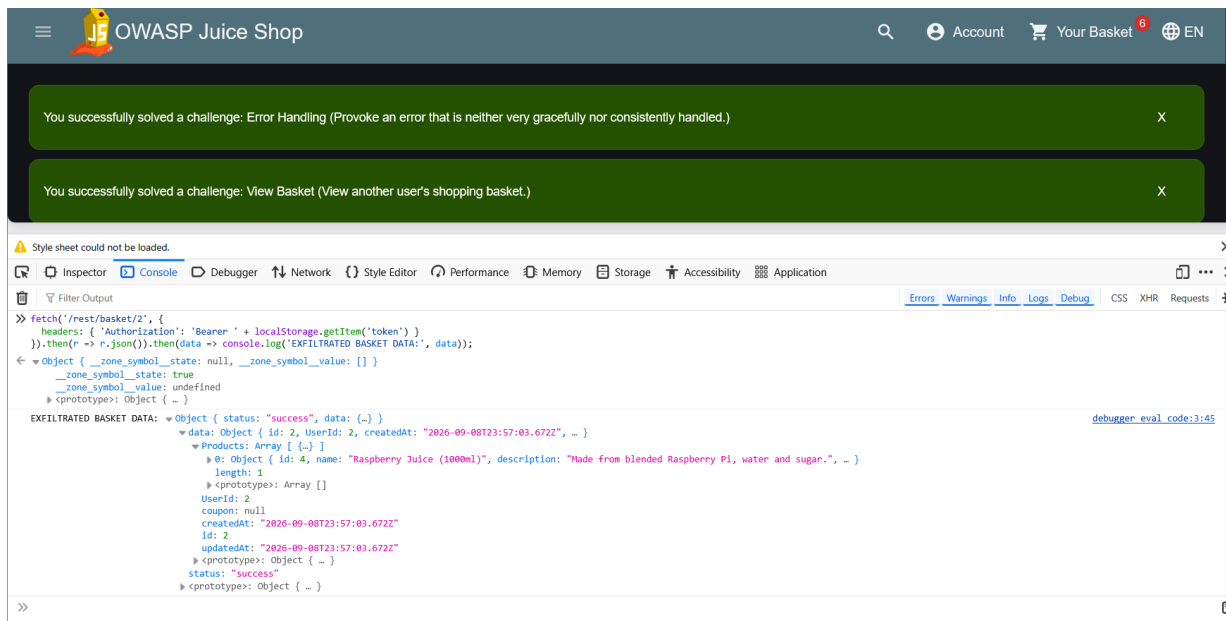
Evidence D2: Exposed API schema details



Source file: Test-4-Information-Disclosure-via-Exposed-Endpoint\Finding4B-Exposed-API-Schema.png

Finding 5: Insecure Direct Object Reference on User Baskets

Evidence E1: Basket data returned for a non-owned resource



Source file: Test-5-Insecure-Direct-Object-Reference-(IDOR) _Broken-Object-Level-Authorization\Finding5-IDOR-Basket-Exfiltration.png

References

- OWASP Juice Shop - <https://owasp.org/www-project-juice-shop/>
- OWASP Web Security Testing Guide v4.2 - <https://owasp.org/www-project-web-security-testing-guide/v42/>
- OWASP Top 10 - <https://owasp.org/Top10/>
- FIRST CVSS v4.0 - <https://www.first.org/cvss/v4.0/>